

UML - CupcakeShop

Documento atualizado para refletir o código atual do projeto PIT II: frontend em HTML, CSS e JavaScript; backend em Python organizado em MVC; persistência em MySQL.

Observação importante: o sistema não possui login/autenticação. O cadastro representa os dados do cliente necessários para finalizar a compra.

1. Visão Geral

O CupcakeShop é uma aplicação web simples para venda de cupcakes. O cliente acessa o catálogo, pesquisa produtos, adiciona itens ao carrinho, preenche o cadastro quando necessário, escolhe a forma de pagamento e finaliza o pedido.

O fluxo atual abre diretamente no catálogo. O cadastro não bloqueia a navegação inicial; ele é solicitado apenas quando o cliente tenta finalizar a compra sem nome, email, telefone e endereço preenchidos.

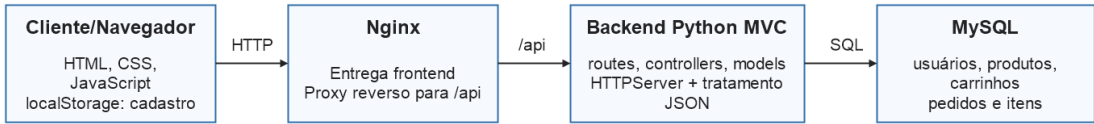
2. Escopo Modelado

Item	Descrição
Ator principal	Cliente
Fora do escopo	Login, painel administrativo, entregador, pagamento real e controle de estoque
Dados do cliente	Nome, email, telefone e endereço preenchidos no frontend e enviados ao backend ao criar pedido
Carrinho	Separado por sessão do navegador usando o cabeçalho X-Session-Id
Histórico visível	Mostra no frontend apenas os pedidos feitos na sessão atual
Persistência	Pedidos, itens, usuários, carrinhos e produtos ficam registrados no MySQL

3. Arquitetura da Aplicação

A aplicação foi organizada de forma simples. O frontend é estático, o Nginx publica os arquivos e encaminha /api para o backend. O backend Python segue separação MVC por rotas, controllers e models.

Diagrama de Componentes - CupcakeShop



Fluxo principal: catálogo inicial -> carrinho -> cadastro se necessário -> pedido salvo no MySQL.

Camada	Arquivos/Pastas	Responsabilidade
Frontend	frontend/index.html, styles.css, script.js	Renderizar telas, controlar navegação, cadastro local, carrinho e consumo da API
Routes	backend/src/routes/router.py	Mapear método e URL para controller
Controllers	backend/src/controllers	Receber contexto HTTP e acionar models
Models	backend/src/models	Aplicar regras de produto, carrinho, usuário e pedido
Database	backend/src/database/connection.py	Abrir conexão e executar consultas MySQL
Config/Utils	backend/src/config, backend/src/utils	Carregar ambiente, formatar JSON e tratar erros

4. Regras do Fluxo Atual

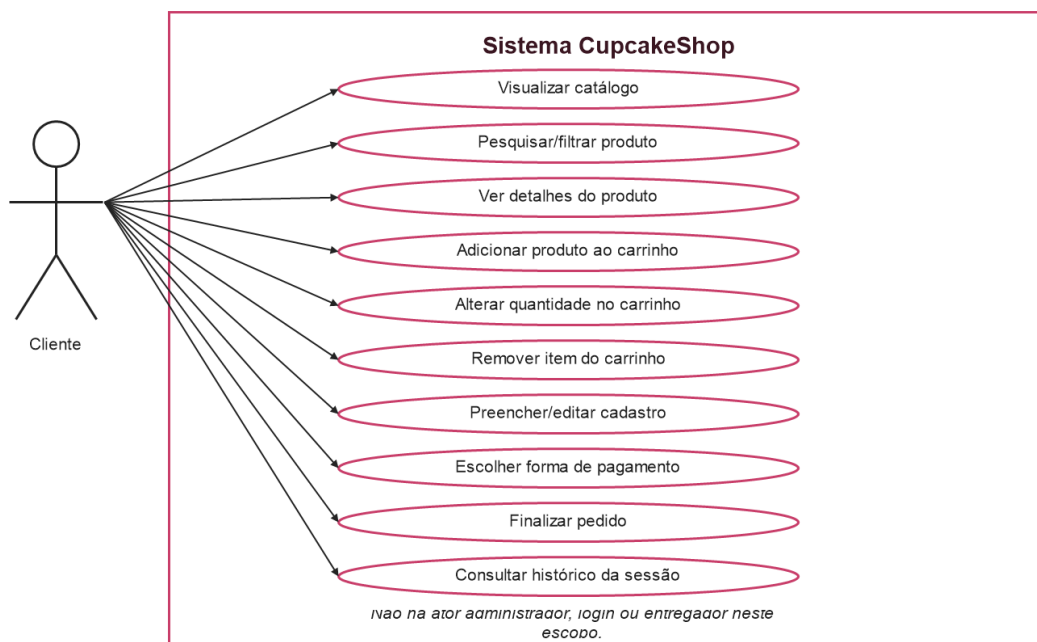
Regra	Como funciona no código atual
Tela inicial	O estado inicial do frontend é route = catalog
Catálogo	GET /api/produtos lista produtos ativos do MySQL
Busca/filtro	Executado no frontend sobre

	os produtos carregados
Carrinho	POST/PATCH/DELETE em /api/carrinho/itens usando X-Session-Id
Contador do carrinho	Soma as quantidades dos itens selecionados
Cadastro	Salvo no localStorage do navegador; campos obrigatórios: nome, email, telefone e endereço
Finalização	Se cadastro estiver incompleto, o frontend envia o cliente para a tela Cadastro
Pedido	POST /api/pedidos envia endereço, pagamento e usuário; backend cria/atualiza usuário por email
Histórico	Frontend mostra os pedidos criados na sessão atual; o banco guarda todos os pedidos persistidos

5. Diagrama de Casos de Uso

O diagrama apresenta as funcionalidades acessadas pelo cliente. Ele reflete o fluxo simples pedido para o projeto: catálogo, carrinho, cadastro no fechamento e pedido.

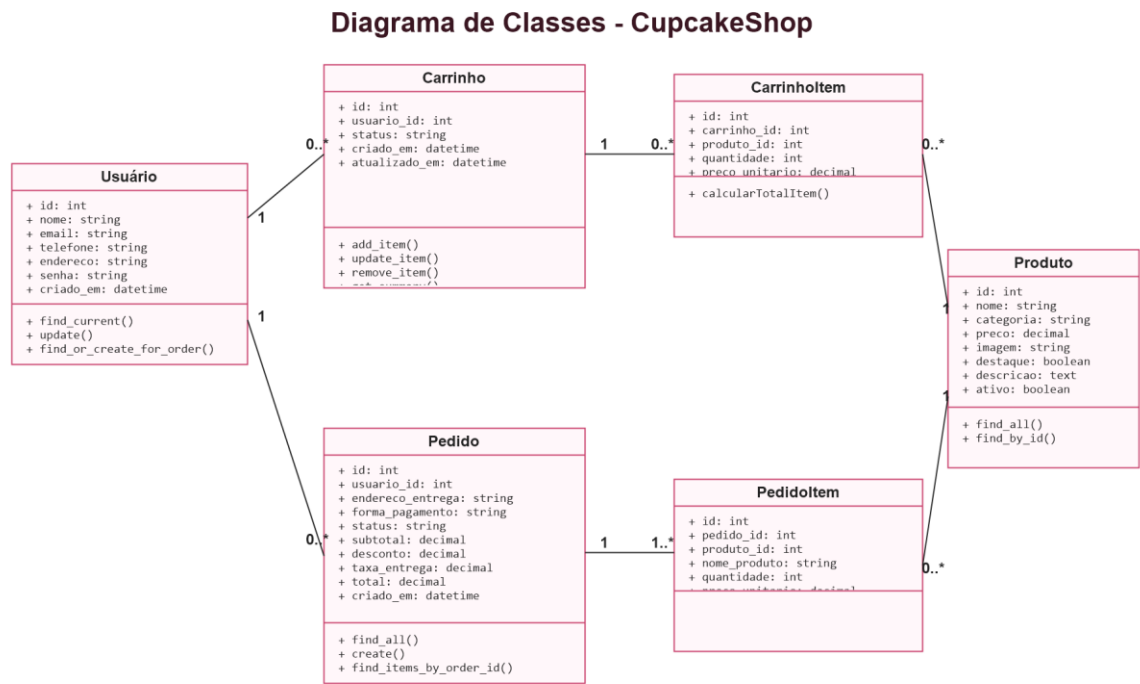
Diagrama de Casos de Uso - CupcakeShop



Caso de uso	Descrição	Implementação principal
Visualizar catálogo	Cliente abre a loja e vê produtos disponíveis	GET /api/produtos
Pesquisar/filtrar produto	Cliente busca ou filtra cupcakes no catálogo	frontend/script.js
Ver detalhes do produto	Cliente abre a tela de detalhes de um cupcake	GET /api/produtos/:id e frontend
Adicionar produto ao carrinho	Cliente adiciona uma ou mais unidades	POST /api/carrinho/itens
Alterar quantidade no carrinho	Cliente aumenta ou diminui unidades	PATCH /api/carrinho/itens/:produtoid
Remover item do carrinho	Cliente remove um produto	DELETE /api/carrinho/itens/:produtoid
Preencher/editar cadastro	Cliente informa nome, email, telefone e endereço	localStorage no frontend; envio em POST /api/pedidos
Escolher forma de pagamento	Cliente seleciona Pix, cartão ou dinheiro	frontend/script.js
Finalizar pedido	Sistema valida cadastro, salva usuário/pedido/itens e limpa carrinho	POST /api/pedidos
Consultar histórico da sessão	Cliente vê pedidos feitos no navegador atual	sessionStorage no frontend

6. Diagrama de Classes

As classes abaixo representam as entidades do domínio e as tabelas persistidas. Como o backend foi implementado em Python simples, os métodos indicados correspondem às principais funções dos models.

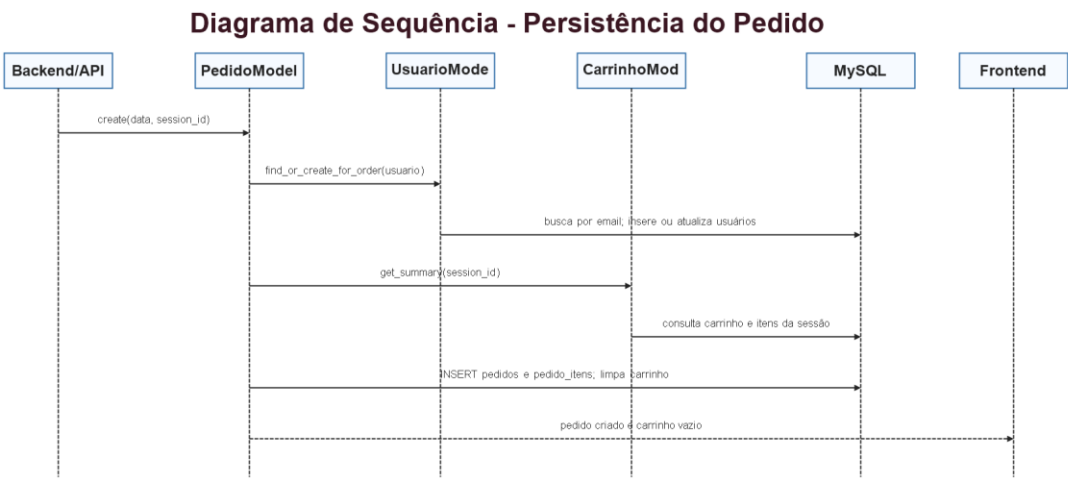
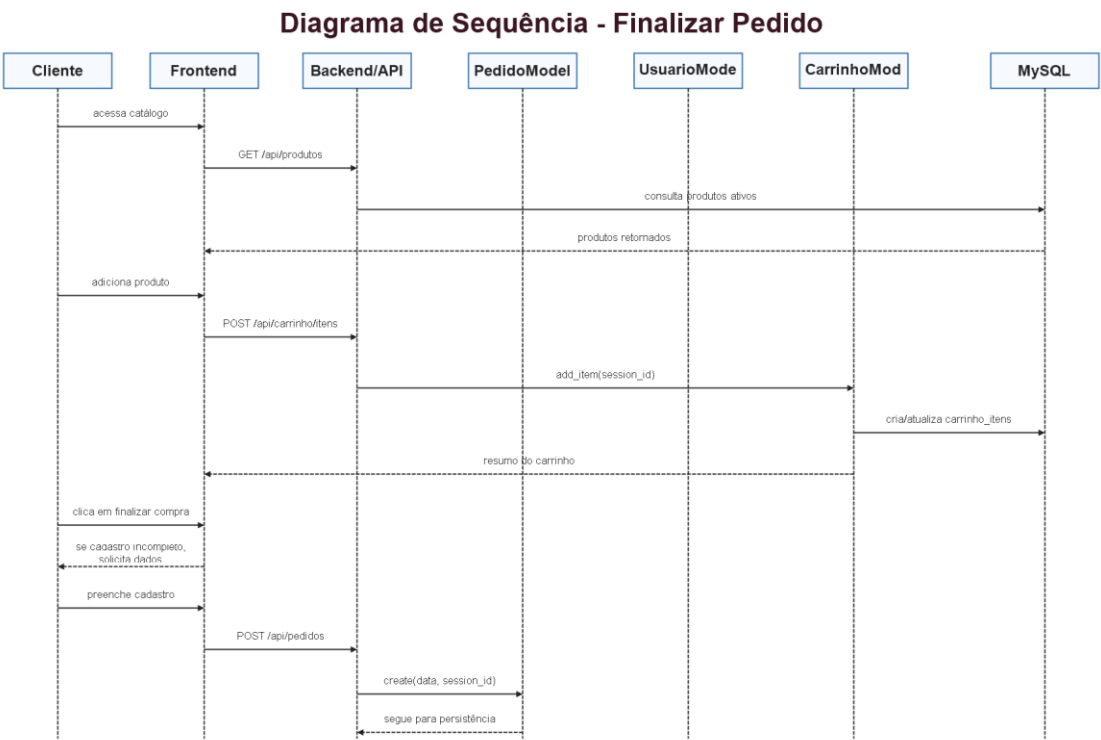


As classes representam as entidades do domínio persistidas no MySQL e os principais métodos dos models Python.

Cardinalidades principais: um usuário pode ter vários carrinhos e pedidos; um carrinho possui vários itens; um pedido possui um ou mais itens; produto aparece nos itens de carrinho e pedido.

7. Diagrama de Sequência

O fluxo de sequência foi dividido em duas imagens para manter legibilidade. A primeira mostra catálogo, carrinho e envio do pedido; a segunda detalha a persistência do pedido no backend.



8. Diagrama de Atividades

O diagrama de atividades mostra o comportamento atual esperado pelo usuário: entrar no catálogo, escolher produtos, ajustar carrinho, preencher cadastro se necessário e finalizar.



9. Relação com o Banco de Dados

Classe/Entidade	Tabela MySQL	Campos principais
Usuário	usuarios	id, nome, email, telefone, endereço, senha, criado_em
Produto	produtos	id, nome, categoria, preco, imagem, destaque, descricao, ativo, criado_em
Carrinho	carrinhos	id, usuario_id, status, criado_em, atualizado_em

CarrinhoItem	carrinho_itens	id, carrinho_id, produto_id, quantidade, preco_unitario, criado_em
Pedido	pedidos	id, usuario_id, endereco_entrega, forma_pagamento, status, subtotal, desconto, taxa_entrega, total, criado_em
PedidoItem	pedido_itens	id, pedido_id, produto_id, nome_produto, quantidade, preco_unitario, total

Relacionamento	Chave estrangeira	Cardinalidade
Usuário -> Carrinho	carrinhos.usuario_id	1 para 0..*
Carrinho -> CarrinhoItem	carrinho_itens.carrinho_id	1 para 0..*
Produto -> CarrinhoItem	carrinho_itens.produto_id	1 para 0..*
Usuário -> Pedido	pedidos.usuario_id	1 para 0..*
Pedido -> PedidoItem	pedido_itens.pedido_id	1 para 1..*
Produto -> PedidoItem	pedido_itens.produto_id	1 para 0..*

10. Relação com o Backend

Funcionalidade	Rota	Controller/Model	Observação
Health check	GET /api/health	router.py	Indica se o banco está configurado
Listar produtos	GET /api/produtos	product_controller / product_model.find_all	Lista produtos ativos
Ver produto	GET /api/produtos/:id	product_controller / product_model.find_by_id	Consulta produto por id
Consultar usuário padrão	GET /api/usuario	user_controller / user_model.find_current	Rota auxiliar mantida no backend
Atualizar usuário padrão	PUT /api/usuario	user_controller / user_model.update	Rota auxiliar; o fluxo atual salva cadastro local e envia no pedido
Consultar carrinho	GET /api/carrinho	cart_controller / cart_model.get_summary	Usa X-Session-Id
Adicionar item	POST /api/carrinho/itens	cart_controller / cart_model.add_item	Cria ou soma item
Alterar item	PATCH /api/carrinho/itens/:produtoid	cart_controller / cart_model.update_item	Altera por delta ou quantidade
Remover item	DELETE	cart_controller /	Remove produto do carrinho

	/api/carrinho/itens/:produtoId	cart_model.remove_item	
Listar pedidos	GET /api/pedidos	order_controller / order_model.find_all	Consulta pedidos persistidos; histórico visível do frontend e da sessão
Criar pedido	POST /api/pedidos	order_controller / order_model.create	Cria/atualiza usuário, grava pedido e itens, limpa carrinho

11. Pontos de Alinhamento com o Código Atual

Ponto revisado	Situação no documento
Frontend abre no catálogo	Representado na visão geral, regras, componente e atividade
Cadastro antes de finalizar	Representado nos casos de uso, sequência e atividade
Sem tela de login	Declarado como fora do escopo
Carrinho por sessão	Representado nas regras, backend e banco por status aberto_sessao
Pedido com vários itens	Representado por Pedido 1 para 1..* PedidoItem
Um pedido aparece em várias linhas no JOIN SQL	Explicado pela tabela pedido_itens; não significa vários pedidos
Histórico apenas da sessão no frontend	Declarado nas regras e casos de uso
Backend Python MVC	Representado na arquitetura e relação com backend

12. Conclusão

A modelagem UML está alinhada ao funcionamento atual do CupcakeShop. O documento cobre os principais diagramas UML necessários para explicar o sistema, relaciona as entidades ao banco MySQL, mostra as rotas implementadas no backend Python MVC e registra as limitações assumidas para manter o projeto simples e adequado ao PIT II.